ISA 401: Business Intelligence & Data Visualization

03: R Foundations

Fadel M. Megahed, PhD

Raymond E. Glos Professor in Business
Farmer School of Business
Miami University

@Fadel Megahed
fmegahed
fmegahed@miamioh.edu
Automated Scheduler for Office Hours

Fall 2026

Before We Start

1 Join today's Zoom

Use the Zoom link in the **Week 0 Module** on Canvas. **Mute** your mic and **turn off** your video.
Log in from the **lab's computer** so you can **share your screen** if you need help with an error.

2 Get your repository ready

Open **GitHub Desktop**. If `business_intelligence` is listed, click **Fetch origin**.
Not on this computer? **File → Clone repository** into **M:\isa401**, exactly like Wednesday.

3 Open RStudio

Just RStudio for now: type *RStudio* in the **Windows search bar**.
RStudio is installed on the lab computers and should already be on your personal laptop from Assignment 0.

4 Get ready for Kahoot

Open **kahoot.it** on your phone or in a browser tab.
The game PIN appears on the next slide: a Week 01 review with a small bonus for the winner.

Done early? **Help the person next to you.** Stuck? **Share your screen on Zoom** and we fix it together.

Week 01 Review on Kahoot

How to play

1. Go to **kahoot.it** on your phone or in a browser tab.
2. Enter the **game PIN** shown on the projector.
3. Use your **first and last name** so the bonus can find you.
4. **20 seconds** per question. Faster correct answers score more.

What is in it

6 questions covering Week 01:

- 3 Class 01: the analytics journey, the shapes of data, visual encodings
- 3 Class 02: commit vs push, diffs, undo vs revert

The winner 🏆 receives a **0.15 bonus** on Assignment 02.

The competition has **no effect on your grade** beyond the bonus. It tells you which Week 01 ideas need another look, and it tells me how to pace today.

Learning Objectives for Today's Class

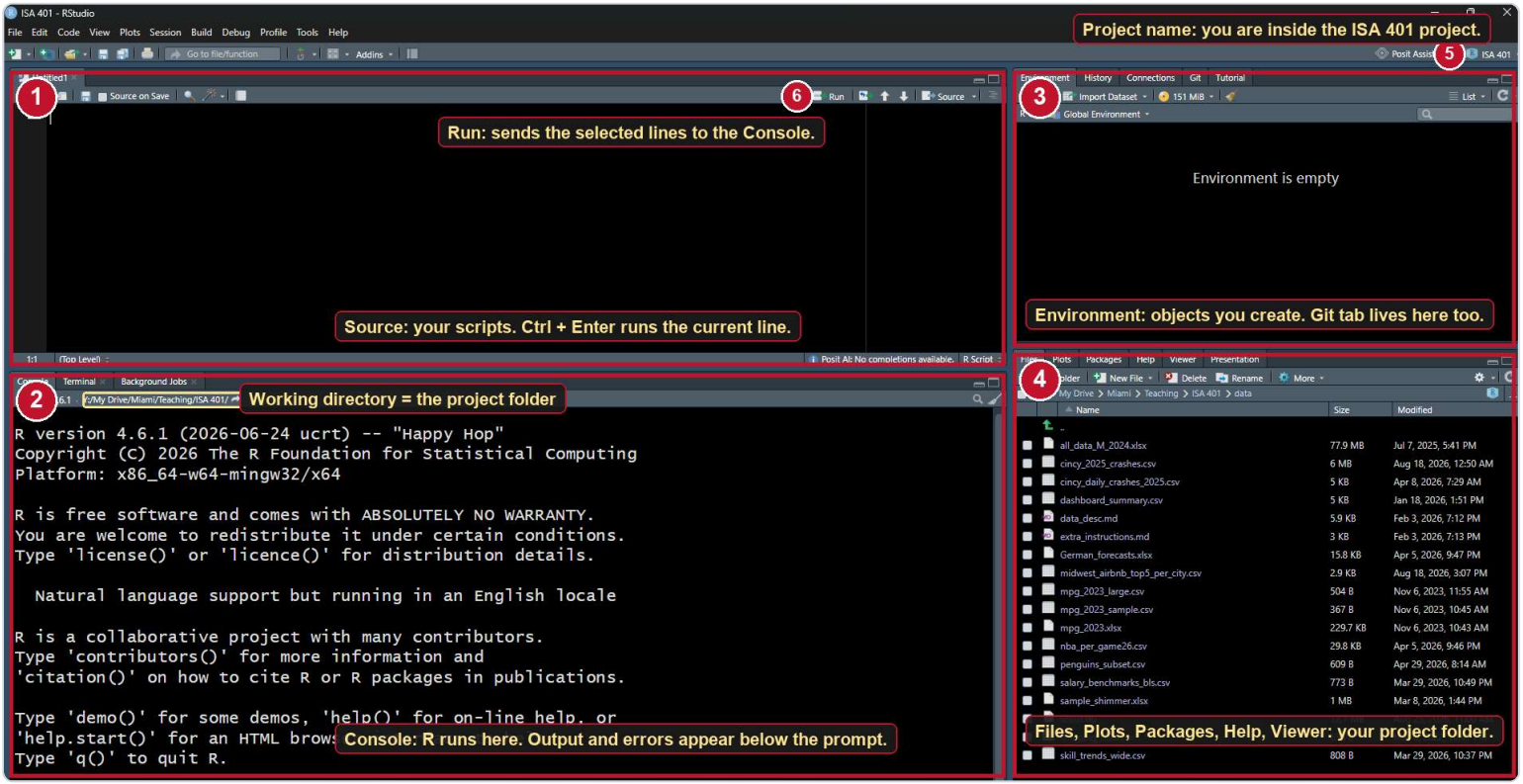
- **Set up** an **RStudio project** inside your repository and work in **R Markdown**.
- **Create and inspect** objects and **vectors** with `=`, `c()`, `names()`, and `typeof()`.
- **Predict** how R handles **types**: one vector, one type, and a comparison returns a logical vector.

PART 1	PART 2
Your R project	**Objects and vectors**
RStudio inside your repo	Assignment, one vector one type

Part 1 is the setup we did not reach on Wednesday. After it, everything is built from seven crash counts you type yourself. No files to download. Data frames, lists, and real data files continue on Wednesday.

Part 1: Your R Project

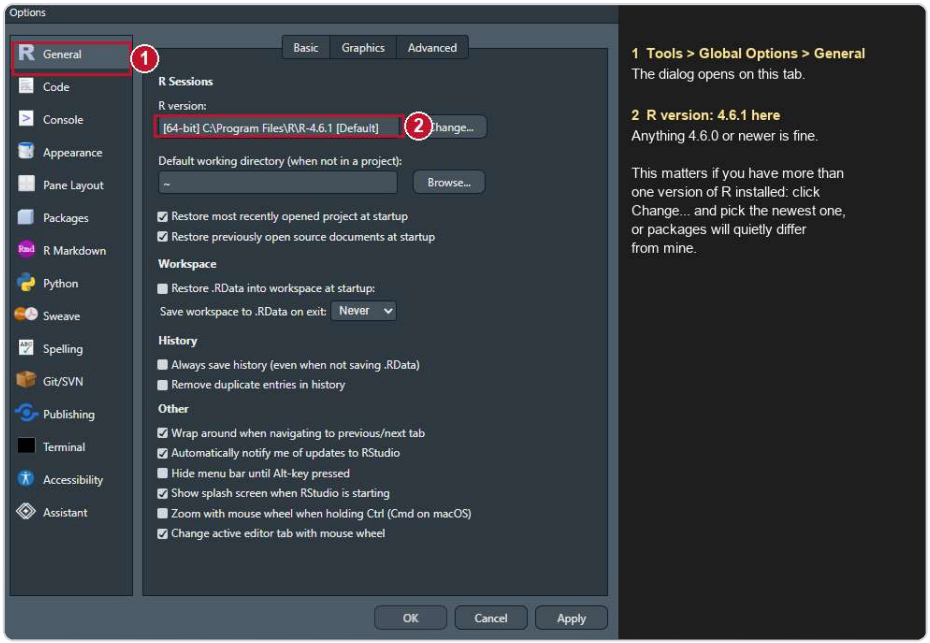
The RStudio Interface



1 Source: your R Markdown files 2 Console: R runs here 3 Environment (and the Git tab) 4 Files, Plots, Packages, Help, Viewer 5 Project name 6 Run

Set Up RStudio Once

1 R version | 2 Workspace | 3 Pipe | 4 Highlighting | 5 Appearance



General → R version

Should read **4.6.0 or newer**. This one is 4.6.1.

Why it matters: if a computer has more than one R installed, RStudio may be pointed at an old one. Click **Change...** and pick the newest, or your packages and results will quietly differ from mine.

What Is a Project?

A self-contained folder

Data, code, and output for one piece of work live together in one folder. Your `business_intelligence` repo is that folder.

The `.Rproj` file marks the root

Open it and RStudio sets the **working directory** to that folder, every time, on any computer. No `setwd()`, ever.

Paths are relative to the root

Write `"data/crashes.csv"`, never `"C:/Users/you/Desktop/crashes.csv"`. The first works after a clone on any machine, including mine when I grade.

YOUR REPO BY THE END OF TODAY

`business_intelligence/` the repo, the project, the folder

- `README.md` from Assignment 0
- `business_intelligence.Rproj` RStudio adds this
- `.gitignore` and this: files Git should skip
- `markdowns/`
  - `class03.Rmd` your first R Markdown
- `data/` Wednesday

Same structure as the course repo you saw in Class 01, just smaller.

Live Demo: From Repo to RStudio Project

Steps | Checkpoint | Discussion

1 Make it a project. RStudio → **File** → **New Project** → **Existing Directory** → Browse to `M:\isa401\business_intelligence` → **Create Project**. RStudio restarts inside the folder; the project name appears top right.

2 Start an R Markdown file. File → **New File** → **R Markdown...**, title *Class 03*, your name, **OK**. Delete everything below the setup chunk, insert a code chunk with **Ctrl + Alt + I**, type the two lines below, then press **Ctrl + Enter** on each one and watch the Console and the Environment.

Copy

```
crashes_per_day = 30125 / 365
crashes_per_day
```

3 Save it inside the project, then knit. Ctrl + S. In the dialog, click **New Folder**, name it `markdowns`, open it, and save the file as `class03.Rmd`. Click **Knit**: `class03.html` appears next to it, your code and output as a web page.

4 Commit and push in GitHub Desktop. Four new items are listed and ticked: `business_intelligence.Rproj`, `.gitignore`, `markdowns/class03.Rmd`, `markdowns/class03.html`. Summary: `Add RStudio project and first markdown` → **Commit to main** → **Push origin**.

5 Verify. On GitHub: one more commit than before, a `markdowns` folder, and your `class03.Rmd` readable in the browser.

IF YOU FALL BEHIND

Say so, or **share your screen on Zoom**.

No **Existing Directory** option? You are already inside a project: **File** → **Close Project** first.

Desktop lists `.Rproj.user/`? It should not; that folder is in `.gitignore`. Untick it if it appears.

Console says `Error: object 'crashes_per_day' not found`? Run line 1 before line 2, or re-run both.

Every Class From Here

1 Open the project

Double-click `business_intelligence.Rproj`, or in RStudio use **File → Open Project**.

Check the top right corner before you type anything.

→

2 Work in a markdown

One R Markdown per class, saved under `markdowns/`, knitted before you push. Relative paths only.

The Console is for trying things; the markdown is the record.

→

3 Commit and push before you leave

GitHub Desktop: tick, summary, **Commit to main**, **Push origin**.

On your laptop: **clone** once (first time only, into any folder you like), then **Pull origin** and the work is there. Worked at home? Push from the laptop, then Pull on the lab PC.

If it is not pushed, it is not done. Your `business_intelligence` repo should contain all your skills from this class.

Part 2: Objects and Vectors

Objects and Assignment

```r
# = stores the value under a name
crashes_per_day = 30125 / 365

# typing the name shows it
crashes_per_day

# <- does exactly the same thing as =
crashes_per_day <- 30125 / 365

# -> also works, pointing the other way
30125 / 365 -> crashes_per_day
```

```
[1] 82.53425
```

Three parts

Name on the left: `crashes_per_day`. Letters, digits, `_` and `.`; no spaces; case matters.

Assignment in the middle: `=`, `<-`, or `->`. We write `=`. You will see `<-` in most R code online and in my editor, and `->` mostly at the end of a pipeline. All three do the same job.

Value on the right: whatever the expression works out to.

Where did it go?

Look at the **Environment** pane (callout 3 on the RStudio slide): `crashes_per_day` is listed with its value. That pane is the list of names R currently knows.

Object 'object_name' not found

1 Case

```
Crashes_per_day
```

```
Error:
! object 'Crashes_per_day' not found
```

2 Typo

```
crashes_per_dya
```

```
Error:
! object 'crashes_per_dya' not found
```

3 Never ran

```
# pretend line 1 was never run:
rm(crashes_per_day)
crashes_per_day
```

```
Error:
! object 'crashes_per_day' not found
```

The same message, three causes. The name on that line is not in the Environment pane. Check, in order:

- capitalization
- spelling
- whether you pressed **Ctrl + Enter** (runs the line) or just **Enter** (adds a new line in the file and runs nothing)

R Is a Vector Language

```r
crashes_by_day = c(2020, 2241, 2327, 2285,
                   2380, 1866, 1664)
names(crashes_by_day) = c("Mon", "Tue", "Wed", "Thu",
                          "Fri", "Sat", "Sun")
crashes_by_day
```

```
 Mon  Tue  Wed  Thu  Fri  Sat  Sun
2020 2241 2327 2285 2380 1866 1664
```

```r
crashes_by_day > 2000        # seven answers
```

```
  Mon   Tue   Wed   Thu   Fri   Sat   Sun
 TRUE  TRUE  TRUE  TRUE  TRUE FALSE FALSE
```

```r
sum(crashes_by_day > 2000)    # days above 2,000
```

```
[1] 5
```

```r
mean(crashes_by_day)
```

```
[1] 2111.857
```

The seven weekday counts from Wednesday's chart

`c()` **combines** values into a **vector**: one type, in order, optionally with names.

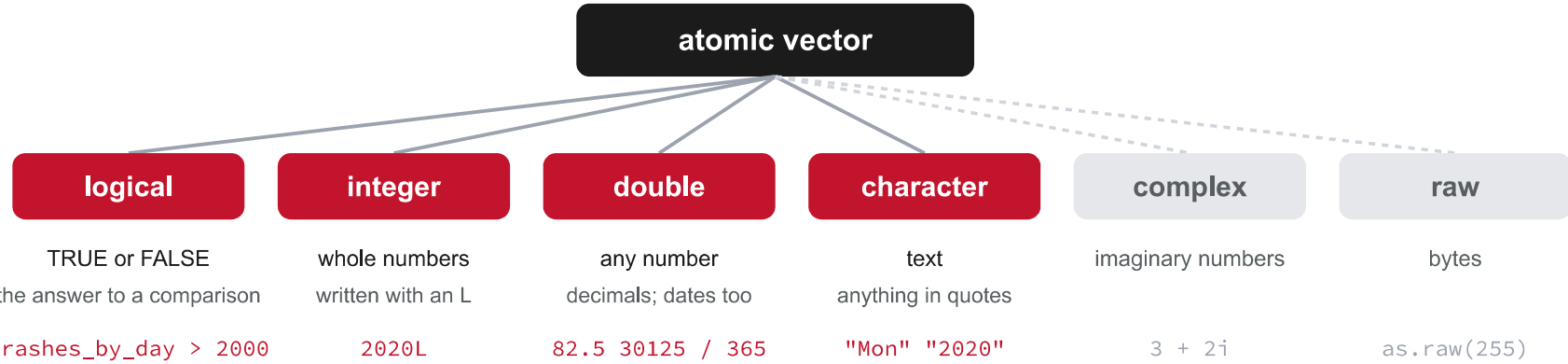
An operation on a vector runs on **every element**: `> 2000` returned seven answers, not one.

`sum()` of seven TRUE/FALSE values counts the TRUEs.

There are no scalars in R. `mean(crashes_by_day)` returned a vector of length 1. Even `82.5` on the last slide was a vector of length 1. A column of a data frame is a vector; that is why this matters.

Six Types of Atomic Vectors

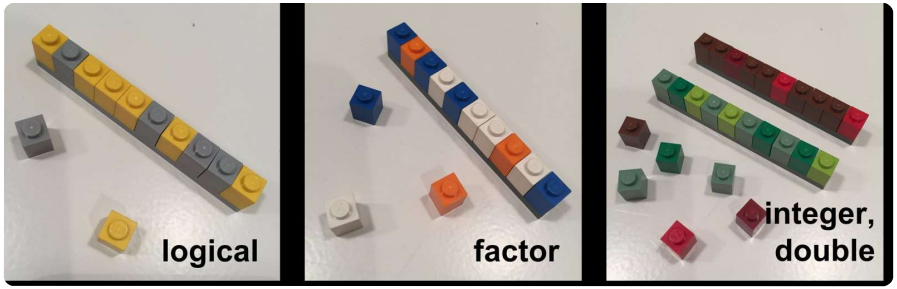
Every vector has exactly one of six types. The first four are the most common in practice.



Red: four data types seen in ISA 401. **Gray:** valid types you are unlikely to need.

Source: the six types follow Advanced R, chapter 3 (Vectors) by Hadley Wickham.

Four Data Types in Lego




```r
typeof(TRUE)      # logical
typeof(401L)      # integer: the L
typeof(401)       # double: no L
typeof("401")     # character: quotes
```

```
[1] "logical"
[1] "integer"
[1] "double"
[1] "character"
```

One vector, one type

Every brick in a row is the same color: a vector holds **one type**. Mix a number into a character vector and R quietly turns everything into character.

`typeof()` tells you which one you have. The *factor* row (categories with fixed levels) arrives with tidy data in week 6.

Image: Jenny Bryan, lego-rstats.

A Self-Assessment for Type Checking

02:00

Game | Connect | Answer | Discussion

Six columns from the Cincinnati crash file, seven rows drawn at random. For every column: **logical, integer, double, or character?** Connect them on the next tab.

instanceid	crashdate	dayofweek	age	latitude_x	injuries
46619FB1	2025-11-10 08:53:00	MON	47	39.138116	4 - POSSIBLE INJURY
0307C584	2025-09-08 15:50:00	MON	32	39.160639	5 - NO APPARENTY INJURY
09803D2E	2025-07-27 12:24:00	SUN	23	39.137162	5 - NO APPARENTY INJURY
A0583304	2025-10-24 20:40:00	FRI	32	39.128541	5 - NO APPARENTY INJURY
2DBB2FF7	2025-09-12 12:42:00	FRI	BB	39.08662	5 - NO APPARENTY INJURY
F9B159B9	2025-09-20 14:52:00	SAT	45	39.101339	5 - NO APPARENTY INJURY
7B5B1C68	2025-10-06 10:18:00	MON	27	39.196296	5 - NO APPARENTY INJURY

`instanceid` is shortened to its first 8 characters to fit. `NA` means the value is missing. `crashdate` is understood as a date when using `readr::read_csv()`.

17 / 22

Recap

Summary of Main Points

By now, you should be able to do the following:

- **Set up** an RStudio project in your repository: open it, work in `markdowns/`, knit, push before you leave.
- **Build and inspect** a named **vector** with `c()` and `names()`; check its type with `typeof()`.
- **Predict** type behavior: one vector, **one type**; mix in text and everything becomes character.
- **Read** a comparison as a **logical vector**: `crashes_by_day > 2000` is seven answers, and `sum()` counts the TRUEs.

Skills practiced: watching the Environment pane, checking `typeof()`, and the type self-assessment on real crash columns.

Wednesday: data frames, lists, subsetting, and your first real data files (CSV, Excel, and a live API).

Before You Go

What is a concept that you would like to l

Review and Clarification

- **Class Notes**: Take some time to revisit your class notes for key insights/concepts.
- **Zoom Recording**: The recording of today's class will be made available on Canvas approximately 3-4 hours after the session ends.
- **AI Recaps:** make your own with Gemini's NotebookLM, as I did for Class 01 (see the right).
- **Questions**: Do not let confusion pile up.

Make your own recap in NotebookLM

- Go to notebooklm.google.com and sign in with your Miami Google account.
- Add today's slide deck as a source: paste the deck's address from Canvas as a **Website** source.
- Click **Audio Overview** for a podcast, or **Video Overview** for a narrated video. Both take a few minutes to generate.

What it looks like: the Class 01 video and podcast. Treat them as a study aid, not a substitute for the slides: the AI summarizes what is on the page and can misstate details.

🎯 Assignments 🎯

Assignment 02: Make the Portfolio Yours

Clone your `portfolio` repo, make it an RStudio project, then edit, commit, and push until every sentence is yours. Submit the live site's address. Due date on Canvas.

Assignment 03: Git, RStudio, and Your Portfolio

A short Canvas quiz on what you did in Assignment 02. Take it after your portfolio edits are pushed. Due date on Canvas.

Before you leave: knit, then commit and push `markdowns/class03.Rmd` and `class03.html` from GitHub Desktop. If it is not pushed, it is not done.

Practice, not graded

1. Make a named vector `injuries_by_day` with any seven numbers and the weekday names.

2. Use one comparison and `sum()` to count how many days in `crashes_by_day` are below 2,000.

3. Predict `typeof(c(2020, "Mon"))`, then run it. Why?

4. Predict `typeof(crashes_by_day > 2000)`, then run it.

5. In one line, compute crashes per hour for every day of the week.

▶ **Answers**